Spectre: Continuous Refinement and Closed-Loop Verification for Rust

Anand Kumar Keshavan[0009−0007−8541−5203]

Independent Researcher, India akkeshavan@gmail.com

Abstract. We present Spectre, a formal-methods toolchain that treats the Rust–specification *refinement relation* as a persistent, executable artifact: derived once from Rust source and maintained across semantic change, verification, repair, testing, and runtime assurance. `spectre mine` extracts a typed model and a machine-readable refinement map from Rust ASTs; `spectre sync` maintains the map under code change via SMT-backed semantic equivalence (Z3/QF_LIA), distinguishing genuine semantic drift from logically-equivalent rewrites. When BFS finds a safety violation, delta-debugging minimises the trace to a 1-minimal sub-sequence, WP substitution synthesises a `require` guard without grammar search, and the guard is translated back to a Rust patch via the refinement map. An ample-set-style POR heuristic and incremental re-verification complete the scalability story. We evaluate the complete lifecycle on *Simplified Raft Leader Election*: exhaustive BFS of 22,432 states (12.4 s), POR heuristic reduces to 6,818 states (7.6× speedup), cache-warm re-run in 0.265 s (47×), one fault fully repaired and one safety-only repair correctly identified, and simulation scaling to a 7-node configuration not exhaustively explored by BFS. Four supporting experiments validate semantic sync precision (AST vs. SMT ablation), repair across five defect classes (4/5 fully synthesised), spec mining across nine author-written components in the supported Rust subset (100% yield), and fault-detection efficiency of systematic over random search (29 distinct violation traces found in 204 explored states vs. 0 in 150,000 random steps).

Keywords: Formal refinement · Semantic synchronisation · CEGIS · WP repair · POR · Model-based testing · Rust

1 Introduction

Formal verification pays off at scale—AWS reports TLA+ catching subtle distributed errors before deployment [20]. Yet adoption stays narrow. The barrier is not expressiveness but a three-stage ergonomics gap: *construction* (authoring a model from scratch), *evolution* (keeping model and code in step as both change), and *remediation* (translating a model-checker verdict into a source fix). Existing workflows require separately authored specifications, mappings, and repair artifacts across these stages. TLA+/TLC [15, 14] and Quint [11] are specification-first; Quint Connect [12] provides a Rust MBT bridge but requires user-supplied

state extraction and step mapping. Ivy [21] and Dafny [17] rely on user-authored formal specifications rather than deriving a Rust–model correspondence from existing source. To our knowledge, no examined workflow derives and reuses a Rust-source correspondence across all of these stages in one artifact.

SPECTRE closes all three gaps by treating the Rust–specification refinement relation as a *persistent executable formal artifact*:

$$\underbrace{\text{derive}}_{\texttt{mine}} \rightarrow \underbrace{\text{verify}}_{\text{BFS/POR}} \rightarrow \underbrace{\text{detect}}_{\text{SMT sync}} \rightarrow \underbrace{\text{diagnose}}_{\text{min. CE}} \rightarrow \underbrace{\text{repair}}_{\text{WP+patch}} \rightarrow \underbrace{\text{re-verify}}_{\text{incremental}} \rightarrow \underbrace{\text{assure}}_{\text{MBT/mon.}}$$

The same `.driver.json` refinement map file drives every stage.

C1: Continuous derivation and semantic maintenance. `spectre mine` extracts a typed Spectre model and `.driver.json` refinement map from Rust ASTs in one pass (zero user annotations). `spectre sync` maintains the map under code change via SMT equivalence (Z3/QF_LIA), distinguishing $x{+}n$ from $x{-}n$ (genuine drift) and $x{>}0$ from $\neg(x{\leq}0)$ (logically equivalent rewrite), and reporting SAT witnesses for every detected semantic change. `spectre impact` maps changed methods to affected invariants for CI gating.

C2: Closed-loop counterexample-to-Rust repair. Delta-debugging [26] reduces violation traces to 1-minimal sub-sequences; causal diagnosis identifies the responsible action. WP substitution [6] then synthesises a `require` guard—no grammar, no search. The guard is translated to a source-level Rust guard patch via the refinement map; the minimised trace becomes a compilable `#[test]` regression test; `spectre verify -incremental` confirms the fix in one incremental pass.

C3: Refinement-derived assurance. The refinement map drives five coverage-guided MBT modes, `proptest` harness generation from action guards (guards → `prop_assume!`; invariants → `prop_assert!`), differential conformance across two spec versions, and inline Rust runtime monitors (<6 ns per check).

C4: Scalable and incremental verification. An ample-set-style POR heuristic selects singleton sets for independent actions to reduce redundant interleavings. Incremental re-verification replaces only the subgraph reachable via a changed action. Property-directed best-first search guides the frontier by linear predicate distance to the nearest invariant boundary. `-param-range` sweeps parameterised models.

Structure. §2 formalises the refinement map. §3 covers change detection and scalable exploration. §4 covers the counterexample-to-repair pipeline. §5 covers assurance outputs. §6 evaluates the full lifecycle on Raft plus four supporting experiments.

2 Refinement Architecture

Kripke structures and safety checking. A Kripke structure $M = (S, S_0, R, L)$ has states, initial states, transition relation, and labelling [4]. Spectre constructs M_Σ by BFS, checking `invariant` clauses on each visited state. Exhaustive termination

is *sound* universal safety verification over all reachable states; when the state limit fires, the result is *no CE found (bounded)*. `eventually(P)` is a bounded witness query, not LTL $M \models \Diamond P$ [22]; full automata-based LTL is future work.

The refinement map. Let Σ_M and Σ_I be the model and implementation state spaces. A *refinement map* $\mathcal{R} \subseteq \Sigma_I \times \Sigma_M$ relates implementation states to model states via field projection π: for $(s_I, s_M) \in \mathcal{R}$, each spec variable equals the corresponding Rust field value. An implementation transition $(s_I \to t_I)$ *conforms* to $\mathcal{R}$ if either: *(i)* there exists a model action A such that $(s_I, s_M), (t_I, t_M) \in \mathcal{R}$, all `require` guards hold, and $s_M \xrightarrow{A} t_M$; or *(ii)* it is a *stuttering step*: $\pi(s_I) = \pi(t_I)$.

`spectre mine` produces a `.spec` model and a `.driver.json` refinement map from struct fields and `impl` methods. Correctness properties (`invariant`) are supplied independently—mining invariants from source would reproduce bugs, not catch them. `spectre propose-invariants` generates candidates from type bounds and recurring guards; each requires explicit human acceptance.

The Spectre language. Variables, guards (`require`), and primed assignments model state transitions. Disabled actions are silently skipped; unmentioned variables are unchanged (frame condition). Listing 1.1 shows the per-node Raft election actions and the cluster-level invariant; Listing 1.2 shows the corresponding Rust.

```
var state1:NodeState; var term1:int; var votes1:int
// (state2..3, term2..3, votes2..3 symmetric)

action startElection1 {
  require state1 != NodeState.Leader; require term1 < 3
  state1' = NodeState.Candidate; term1' = term1 + 1; votes1' = 1
}
action becomeLeader1 {
  require state1 == NodeState.Candidate; require votes1 * 2 > 3
  state1' = NodeState.Leader
}
invariant electionSafety { /* at most one Leader per term */ }
invariant leaderMajority { /* every Leader holds strict majority */ }
```

Listing 1.1. 3-node Raft spec: per-node variables; 3 instances composed to form cluster.

```
pub struct RaftNode { pub state: i64, pub term: i64, pub votes: i64 }
impl RaftNode {
    pub fn start_election(&mut self) {
        assert!(self.state != 2); // not leader
        self.state = 1; self.term += 1; self.votes = 1;
    }
    pub fn become_leader(&mut self) -> Result<(), String> {
        assert!(self.state == 1);
        if self.votes * 2 <= 3 { return Err("no majority".into()); }
        self.state = 2; Ok(())
    }
}
```

Listing 1.2. Raft node in Rust (8 methods, 9 `assert!` guards; 30 LOC).

Figure 1 shows the continuous refinement lifecycle. Table 1 positions Spectre among related tools.

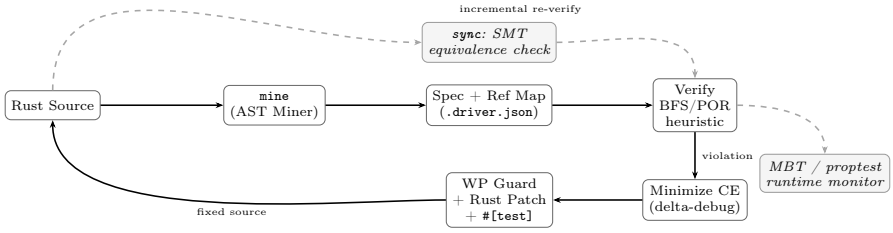

Fig. 1. Spectre continuous refinement cycle. The `.driver.json` map (produced by `mine`) is reused by every downstream stage. The repair loop (violation → minimize → WP patch → fixed Rust) closes at the source level. The dashed change-detection path (SMT sync → incremental re-verify) keeps the map consistent across code evolution.

Table 1. Feature comparison (open-source toolchains).

Feature	TLA+/TLC	Quint	Spectre
Source-derived Rust–model map	–	partial†	✓
SMT semantic sync / drift detection	–	–	✓
CE minimisation (delta-debugging)	–	–	✓
WP guard synthesis + Rust patch	–	–	✓
Ample-set-style POR heuristic	○*	–	✓
Property-directed best-first search	–	–	✓
Incremental re-verification	–	–	✓
Coverage-guided MBT (5 modes)	–	○	✓
`proptest` harness gen.	–	–	✓
Runtime monitor generation	–	–	✓
Infinite-trace LTL / WF / SF	✓	✓	future work

†Quint Connect: user-supplied state extraction required.
*TLC provides symmetry/state-graph reduction; ample-set POR is not documented.

3 Maintaining Refinement Under Change

SMT semantic equivalence (`spectre sync`). `spectre sync` re-mines the Rust source and classifies each changed guard or assignment expression. Structural comparison catches new/renamed variables and actions but cannot distinguish $x+n$ from $x-n$ (genuine semantic change) vs. $x>0$ from $\neg(x\leq 0)$ (logically equivalent). Spectre encodes the query $\exists \boldsymbol{x}.\, e_{old}(\boldsymbol{x}) \neq e_{new}(\boldsymbol{x})$ in QF_LIA and calls Z3 [19]: UNSAT means equivalent; SAT returns a distinguishing witness for the developer.

Proposition 1 (SMT Equivalence Soundness). *If the QF_LIA query is UNSAT, then e_{old} and e_{new} are identical for all integer/Boolean valuations. Any SAT witness is a* distinguishing valuation in the encoded domain*: it proves*

expression inequivalence but need not correspond to a reachable Rust program state.

The encoding uses no `set-logic` declaration when Boolean variables are present (letting Z3 choose); negative literals are encoded as $(-c)$; Boolean variables are `Bool`-sorted compared via `xor`.

PR / CI impact analysis (`spectre impact`). Given a `git diff`, Spectre maps changed Rust method names to affected spec actions, identifies invariants referencing those actions' write-sets, and reports which invariants need re-verification. Output gates CI pipelines: exit 1 on any non-preserving semantic change.

Cached and incremental verification. Spectre SHA-256-hashes the spec, parameters, and interpreter version, caching the BFS graph under `.spectre-cache/`. A cache hit avoids all re-exploration (47× speedup on Raft: 0.265 s vs. 12.4 s cold BFS). When `sync` flags an action as *possibly-unsafe*, `verify -incremental` prunes stale transitions, recomputes reachability, and BFS-expands newly reachable states within the original bound.

Proposition 2 (Incremental Equivalence). *Let $\mathcal{C}$ be a fixed exploration configuration (initial state, parameters, depth/state bounds, action order, interpreter semantics) and $G_{\mathcal{C}}(S)$ the graph produced under $\mathcal{C}$. If S' differs from S only in action A's transition relation, incremental re-verification under $\mathcal{C}$ yields $G_{\mathcal{C}}(S')$.*

Partial-order reduction. Spectre applies a conservative ample-set-style POR heuristic [8]: two actions are *independent* if their read/write variable sets are disjoint; if one action is independent of all others it is explored alone, otherwise the full enabled set is used. We validate POR results against full BFS where feasible; empirically, POR reduces the 3-node Raft model from 22,432 to 6,818 states (7.6× speedup) and processes 5-node Raft 6.9× faster at the 50,000-state bound.

Property-directed best-first search. Each state is assigned a *linear predicate distance* to the nearest invariant violation: $d(e \geq c) = \max(0, c - e)$, $d(e > c) = \max(0, c - e - 1)$, $d(e \leq c) = \max(0, e - c)$, $d(e < c) = \max(0, e - c - 1)$, $d(I_1 \wedge I_2) = d(I_1) + d(I_2)$, $d(I_1 \vee I_2) = \min(d(I_1), d(I_2))$. States are dequeued by ascending distance; the frontier prioritises states nearest invariant boundaries.

4 From Counterexample to Rust Repair

Counterexample minimisation. Given a violation trace of length ℓ, Spectre applies Zeller's ddmin algorithm [26] to find a *1-minimal* sub-sequence: removing any single step destroys the violation. Each candidate subsequence is re-executed from the original initial state under the model's transition semantics; it is retained only if every remaining action is still enabled and the violation still occurs. Path reconstruction uses a linear forward scan through original transitions (preserving relative order after removals), avoiding positional mismatch when the same action appears multiple times.

Algorithm 1 SynthesiseGuard(I, A)

Require: Invariant I; action A with simultaneous assignments $\{x'_i = e_i\}$
Ensure: `require` guard g sufficient to block the violation
1: $g \leftarrow I[\, x_1 \leftarrow e_1,\ \ldots,\ x_n \leftarrow e_n \,]$
2: Simplify g; add as `require` to A; re-explore
3: **if** new violation with failing pair (A', I') **then return** SynthesiseGuard(I', A')
4: **end if**
5: **return** g

WP synthesis. For invariant I and action A with simultaneous assignments $\{x'_1 = e_1, \ldots, x'_n = e_n\}$, the weakest precondition is computed by syntactic substitution [6]:

$$\mathrm{wp}(A, I) = I[\, x_1 \leftarrow e_1,\ \ldots,\ x_n \leftarrow e_n \,]$$

Proposition 3 (Substitution Soundness). *For any pre-state σ in which A is enabled: $\sigma \models \mathrm{wp}(A, I)$ implies $A(\sigma) \models I$. This holds for all assignment forms (arithmetic, Boolean, enum) with no grammar search.*

Algorithm 1 shows the CEGIS loop: a CE identifies the failing (A, I) pair; $\mathrm{wp}(A, I)$ is computed *once* by syntactic substitution (no grammar, no search), yielding a safety repair (invariant preserved; intended semantics not necessarily restored); if re-verification yields a new (A', I'), the procedure recurses on the already-repaired model—guards from earlier iterations remain installed.

Rust patch generation. The synthesised guard g is translated to Rust via the refinement map: spec variable v maps to `self.rust_field` for state variables and to the local parameter name for action arguments. Operator precedence is respected (right-operand wrapping for `-` and `/`). If the refinement map marks the method as returning `Result`, the patch emits `if !(g) { return Err(...); }`; otherwise `assert!(g)`.

Generated regression test. The 1-minimal CE trace is serialised as ITF JSON and `spectre verify -emit-rust-tests` produces a compilable `#[test]` reproducing the fault. After the patch is applied, `spectre verify -incremental` confirms the invariant holds.

5 Generated Conformance and Runtime Assurance

All outputs below are derived from the same `.driver.json` map.

Coverage-guided MBT. `spectre verify -emit-traces -coverage-mode` generates ITF traces in five modes: *action* (prefer unseen actions), *transition-pair* (prefer unseen (A_{prev}, A_{curr}) pairs), *boundary* (steer toward integer variable min/max), *rare-action* (prefer least-frequently-taken transitions), and *property-directed* (minimise linear predicate distance to invariant boundary). Traces are replayed against live Rust via the generated `StepHandler` driver.

Proptest, differential conformance, and runtime monitors. `spectre verify -emit-proptest` translates `require` guards into `prop_assume!` filters and invariants into `prop_assert!`. `spectre diff-conformance` replays one ITF trace against two spec versions, surfacing steps where verdicts diverge. `spectre generate-monitor` produces a self-contained `monitor.rs` checking all invariant predicates in $O(k)$ time with no heap allocation (3.7–6.1 ns per check on Apple M3 Pro, $k = 1$–20 clauses).

6 Evaluation

Platform. Apple M3 Pro (`aarch64`), 18 GB RAM, macOS 15.6, Go 1.24. Default limits: 5,000 states / depth 100 unless noted. All benchmarks are in `examples/`; a Docker image and `artifact/reproduce.sh` enable reproduction. Absolute wall times are hardware-dependent; speedup ratios (BFS/POR, cold/warm, full/incremental) are the primary claims and are reproducible on any platform within normal variance.

We evaluate four research questions: **RQ1** Raft complete lifecycle (derivation, scalability, POR, repair); **RQ2** semantic sync ablation (structural vs. AST vs. SMT); **RQ3** end-to-end repair across defect classes; **RQ4** spec mining breadth and guided-exploration effectiveness.

6.1 RQ1 — Raft: Complete Lifecycle on a Distributed Protocol

Complete lifecycle. We drive the entire Spectre lifecycle on Raft in a single continuous thread:

(1) Derive. `spectre mine raft.rs` extracts a per-node skeleton (5 vars, 5 action types: `startElection`, `grantVote`, `becomeLeader`, `appendEntry`, `stepDown`) and a `.driver.json` in one pass.
(2) Parameterise. Three composed instances form the 3-node cluster spec (15 vars, 27 actions); 5-node (25/75) and 7-node (35/147) follow the same pattern.
(3) Verify + POR heuristic. Exhaustive BFS of the 3-node cluster finds no violations in 22,432 states. POR heuristic reduces to 6,818 states (7.6× speedup).
(4) Incremental. Cache warm re-run: 0.265 s (47×); per-action incremental: 4.3–5.2×.
(5) Inject fault. Wrong quorum (`votes1 >= 1`); BFS detects *leaderMajority* violation.
(6) Minimize CE. Delta-debugging confirms the trace is already 1-minimal (2 steps).
(7) Diagnose. Relevant vars: `state1, votes1`; first divergence at step 1.
(8) WP repair. Guard synthesised: `votes1 * 2 > 3`.
(9) Rust patch. Patch: `assert!(self.votes * 2 > 3)`; compiles without error.
(10) Regression test. Minimised CE serialised as `#[test]`; passes after patch applied.

Table 2. Raft scalability. † State limit reached; full reachable-state count not determined. 5-node POR heuristic is 6.9× faster than BFS at the same budget.

Config	Vars	Actions	States explored	Time	Sim (100 traces)
3-node BFS	15	27	22,432	12.4 s	–
3-node POR	15	27	6,818	1.63 s	–
5-node BFS	25	75	>50,000†	268 s	0.56 s
5-node POR	25	75	>50,000†	38.7 s	–
7-node	35	147	not attempted	–	1.64 s

Table 3. Incremental re-verification on 3-node Raft (cold BFS 12.4 s). *Pruned* = states removed as unreachable without the changed action.

Changed action	Incremental	Speedup	Pruned	New BFS
`stepDown1_s2`	2.4 s	5.2×	1,649	≈1,644
`vote2for1`	2.9 s	4.3×	6,501	≈6,493

(11) Re-verify. `spectre verify -incremental` confirms invariant holds. Total end-to-end time (detect → re-verify): 1.6 s.

(12) Semantic drift + safety repair. Second fault: `term1' = term1` (missing `+ 1`). `spectre sync` flags genuine drift; 6-step CE violates *electionSafety*. WP guard: `term1 != term2 && term1 != term3`—a **safety repair**: restricts executions to preserve the invariant but does not restore the missing increment; `diff-conformance` flags over-restriction. The underlying bug requires a code fix, not a guard.

(13) Scale. 5-node BFS: >50,000 states in 268 s (6.9× faster with POR: 38.7 s); reachable space exceeds the 50,000-state bound—simulation provides coverage (100 traces, 0.56 s). 7-node: simulation only (100 traces, 1.64 s).

Terms and log lengths are bounded 0–3; this is a finite abstraction of the election sub-protocol, not full Raft verification. Tables 2–4 report detailed measurements.

6.2 RQ2 — Semantic Synchronisation Ablation

Setup. We construct two corpora from the `bank_account` Rust component: (a) 8 genuine semantic mutations (wrong operator, wrong comparison, polarity inversion, guard omission); (b) 5 semantics-preserving transformations (commutativity, constant folding, double negation, guard reordering). We compare three methods: field/action count only (*Structural*); normalised AST string matching (*Norm. AST*); and *AST + SMT* (our approach).

Structural comparison detects only count-level changes (3/8 semantic), missing arithmetic mutations. Normalised AST detects all semantic changes but rejects 4/5 equivalent rewrites (high FP). AST + SMT detects all 8 semantic mutations with only 1 FP (guard reordering, a known limitation of positional matching)—a clear win over both baselines.

Table 4. End-to-end repair on injected Raft faults. CE/Min = trace length (original / 1-minimal). †Safety repair: guard restricts executions to preserve the invariant but does not restore the missing `term+1` assignment; `diff-conformance` detects over-restriction.

Injected fault	CE	Min	WP guard	Patch	Test
Wrong quorum	2	2	`votes1*2 > 3`	✓	✓
Missing term incr.†	6	6	`term1≠term2 ∧ term1≠term3`	✓†	✓

Table 5. Semantic sync ablation (8 semantic mutations + 5 equivalent rewrites). FP = false positives (equivalent edits flagged); FN = false negatives (semantic changes missed).

Method	Semantic detected	Equiv. accepted	FP	FN
Structural	3 / 8	5 / 5	0	5
Normalised AST	8 / 8	1 / 5	4	0
AST + SMT	8 / 8	4 / 5	1	0

6.3 RQ3 — End-to-End Repair

We apply the full seven-stage pipeline—CE, minimize, WP, patch, compile, `#[test]`, re-verify—to one representative defect per class.

The unsynthesisable case (`repair-protocol`) illustrates a known limitation: the *missing readiness guard* requires WP reasoning across two actions (the setter and the consumer); single-action WP cannot express it. This is a precise boundary, not a random failure, and is future work.

6.4 RQ4 — Spec Mining and Directed Testing

Mining breadth within the supported Rust subset. We measure construct diversity (not yet external validity) by applying `spectre mine` to 9 author-written Rust components spanning five design archetypes: bounded resource managers (semaphore, connection pool), FIFO/LIFO collections (queue, stack), event-driven structures (event log), finite-state machines (circuit breaker), and key/value stores (cache, bank account, rate limiter). All 9 produce parseable, type-correct specs with zero manual edits—100% field/action yield across primitive types (`int`, `bool`, `enum`). All nine mined specifications parse, type-check, and execute under Spectre without manual modification. Evaluation on independent open-source crates is left to future work.

Generic or `unsafe` Rust requires additional manual abstraction beyond the current miner's scope.

Guided exploration vs. random simulation (RQ4b). A 5-action seeded-fault spec requires 6 consecutive `inc` steps; 3 distractor and 1 reset actions suppress random walks.

Table 6. End-to-end repair pipeline across defect classes. CE/Min = original / minimised trace length; √ = stage succeeded; – = stage not reached.

Spec	CE / Min	WP guard	WP	Patch	Compile	Test	Reverify
`repair-arith`	1/1	`balance - amount >= 0`	√	√	√	√	√
`repair-bool`	1/1	`!(active = true)`	√	√	√	√	√
`repair-enum`	1/1	`status != Running`	√	√	√	√	√
`repair-simultaneous`	4/1	`balance - 30 >= 0`	√	√	√	√	√
`repair-protocol`	2/2	(missing upstream guard)	–	–	–	–	–
Total			4/5	4/5	4/5	4/5	4/5

Table 7. Mining breadth: 9 Rust components, 100% automatic yield. † BFS bounded at 5,000 states.

Component	LOC	Fields	Actions	Guards	BFS / time
`cache`	13	4	4	1	≥5,000† / 0.35 s
`rate_limiter`	11	3	3	3	1 / <0.01 s
`bank_account`	30	3	4	7	3,745 / 1.70 s
`queue`	12	4	3	2	1 / <0.01 s
`circuit_breaker`	14	4	4	5	5 / <0.01 s
`stack`	11	3	3	2	2 / <0.01 s
`semaphore`	12	3	4	4	2,550 / 0.07 s
`event_log`	14	4	5	6	2 / <0.01 s
`connection_pool`	12	3	4	4	1 / <0.01 s

Both systematic strategies find all 29 distinct violation-reaching traces after 204 explored states; none of the 5,000 independent 30-step random simulations finds even one.

7 Related Work

Specification-first tools. TLA+/TLC [15, 14] and Quint [11] require user-authored models. Quint Connect [12] and tla_connect [24] provide Rust MBT and ITF replay bridges, but state extraction and step mapping are user-supplied. Cîrstea et al. [3] validate distributed program traces against TLA+ specs; Maliuginas and Petrauskas [18] extract TLA+ from a BEAM virtual machine program. Spectre pursues the complementary *implementation-first* direction: derivation, synchronisation, repair, testing, and monitoring are all driven by a single source-derived refinement correspondence.

Annotation-based, deductive, and synthesis tools. Ivy [21] and Dafny [17] rely on user-authored formal specifications rather than deriving a Rust–model correspondence from existing source. P [5] unifies implementation and systematic testing in a dedicated event-driven language; Spectre operates on existing idiomatic Rust. CEGIS [23] iteratively refines candidates using counterexamples; SyGuS [1] additionally constrains candidates via a syntactic grammar. Spectre's candidate

Table 8. Systematic search finds all 29 distinct violation-reaching traces after 204 explored states; random simulation finds none in 150,000 steps (5,000 runs × 30 steps, seeds 1–5,000). PD = property-directed.

Strategy	Budget	Violations	States / steps explored
BFS	50,000 states	29	204 states
PD (`-property-guided`)	50,000 states	29	204 states
Random (≤5,000×30)	150,000 steps	0	150,000 steps

guard is instead computed directly by WP substitution and translated back to Rust via the refinement map. Verus [16] verifies Rust via user-authored ghost-type annotations; ExVerus [25] adds CE-guided proof repair. Spectre requires no inline proof annotations or proof obligations; correctness invariants remain independently supplied by the user.

Model extraction, invariant inference, and runtime verification. SPIN/Modex [8, 9] extract Promela from C; Spectre targets Rust and additionally reuses the correspondence for repair, testing, and monitoring. Daikon [7] infers invariants dynamically; Spectre generates candidates requiring explicit human acceptance. MOP/JavaMOP [2, 13] use separate monitor DSLs; Spectre derives monitors from the same spec. ITF traces [10] are the interchange standard.

8 Conclusion

Spectre demonstrates that a source-derived refinement map can serve as persistent verification infrastructure across the Rust development lifecycle. The central thesis—that the refinement relation should be derived once and maintained, not rebuilt per tool—is validated by showing the same `.driver.json` artifact driving semantic sync, incremental verification, CE minimisation, WP patch generation, MBT trace replay, proptest harnesses, and runtime monitors. The Raft case study demonstrates the complete system: 22,432-state exhaustive verification, 7.6× POR heuristic speedup, 47× cache speedup, incremental re-verification (4.3–5.2× speedup), and end-to-end fault repair. Limitations: bounded checking; no infinite-trace LTL; external-crate mining requires manual abstraction for generic/unsafe Rust. The tool, artifact, and Docker image are at `https://github.com/BoundlessProducts/spectre`; artifact DOI: https://doi.org/10.5281/zenodo.2188675210.5281/zenodo.21886752.

References

1. Alur, R., Bodik, R., Juniwal, G., Martin, M.M.K., Raghothaman, M., Seshia, S.A., Singh, R., Solar-Lezama, A., Torlak, E., Udupa, A.: Syntax-guided synthesis. In: Formal Methods in Computer-Aided Design (FMCAD 2013). pp. 1–17. IEEE (2013). https://doi.org/10.1109/FMCAD.2013.6679385

2. Chen, F., Roşu, G.: MOP: An efficient and generic runtime verification framework. In: Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA 2007). pp. 569–588. ACM (2007). https://doi.org/10.1145/1297027.1297069
3. Cîrstea, H., Kuppe, M.A., Loillier, B., Merz, S.: Validating traces of distributed programs against TLA+ specifications. In: Software Engineering and Formal Methods (SEFM 2024). Lecture Notes in Computer Science, vol. 15280, pp. 126–143. Springer (2024). https://doi.org/10.1007/978-3-031-77382-2_8
4. Clarke, E.M., Emerson, E.A., Sistla, A.P.: Automatic verification of finite-state concurrent systems using temporal logic specifications. ACM Transactions on Programming Languages and Systems (TOPLAS) **8**(2), 244–263 (1986). https://doi.org/10.1145/5397.5399
5. Desai, A., Gupta, V., Jackson, E., Qadeer, S., Rajamani, S., Zufferey, D.: P: Safe asynchronous event-driven programming. In: Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2013). pp. 321–332. ACM (2013). https://doi.org/10.1145/2491956.2462184
6. Dijkstra, E.W.: A Discipline of Programming. Prentice-Hall (1976)
7. Ernst, M.D., Perkins, J.H., Guo, P.J., McCamant, S., Pacheco, C., Tschantz, M.S., Xiao, C.: The Daikon system for dynamic detection of likely invariants. Science of Computer Programming **69**(1–3), 35–45 (2007). https://doi.org/10.1016/j.scico.2007.01.015
8. Holzmann, G.J.: The SPIN Model Checker: Primer and Reference Manual. Addison-Wesley (2004)
9. Holzmann, G.J., Smith, M.H.: Software model checking: Extracting verification models from source code. Software Testing, Verification and Reliability (STVR) **11**(2), 65–79 (2001). https://doi.org/10.1002/stvr.228
10. Informal Systems: The informal trace format (ITF). Apalache Architecture Decision Record ADR-015 (2022), `https://apalache.informal.systems/docs/adr/015adr-trace.html`
11. Informal Systems: Quint: A modern specification language for TLA+. GitHub Repository (2023), `https://github.com/informalsystems/quint`
12. Informal Systems: Quint connect: Rust integration for Quint. GitHub Repository Module (2024), `https://github.com/informalsystems/quint/tree/main/quint-connect`
13. Jin, D., Meredith, P.O., Lee, C., Roşu, G.: JavaMOP: Efficient parametric runtime monitoring framework. In: Proceedings of the 34th International Conference on Software Engineering (ICSE 2012). pp. 1427–1430. IEEE (2012). https://doi.org/10.1109/ICSE.2012.6227231
14. Kuppe, M.A., Lamport, L., Ricketts, D.: The TLA+ toolbox. Electronic Proceedings in Theoretical Computer Science (EPTCS) **310**, 50–62 (2019). https://doi.org/10.4204/EPTCS.310.6
15. Lamport, L.: Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. Addison-Wesley (2002), `https://www.microsoft.com/en-us/research/publication/specifying-systems-the-tla-language-and-tools-for-hardware-and-software-engineers/`
16. Lattuada, A., Hance, T., Cho, C., Brun, M., Subasinghe, I., Zhou, Y.Z., Howell, J., Parno, B., Hawblitzel, C.: Verus: Verifying Rust programs using linear ghost types. Proceedings of the ACM on Programming Languages (PACMPL) **7**(OOPSLA1), 1–30 (2023). https://doi.org/10.1145/3586037

17. Leino, K.R.M.: Dafny: An automatic program verifier for functional correctness. In: Logic for Programming, Artificial Intelligence, and Reasoning (LPAR 2010). Lecture Notes in Computer Science, vol. 6355, pp. 348–370. Springer (2010). https://doi.org/10.1007/978-3-642-17511-4_20
18. Maliuginas, A., Petrauskas, K.: Extracting TLA+ specifications out of a program for a BEAM virtual machine. Vilnius University Open Series pp. 106–114 (2024). https://doi.org/10.15388/LMITT.2024.14
19. de Moura, L., Bjørner, N.: Z3: An efficient SMT solver. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008). Lecture Notes in Computer Science, vol. 4963, pp. 337–340. Springer (2008). https://doi.org/10.1007/978-3-540-78800-3_24
20. Newcombe, C., Rath, T., Zhang, F., Munteanu, B., Brooker, M., Deardeuff, M.: How Amazon Web Services uses formal methods. Communications of the ACM **58**(4), 66–73 (2015). https://doi.org/10.1145/2699417
21. Padon, O., McMillan, K.L., Panda, A., Sagiv, M., Shoham, S.: Ivy: Safety verification by interactive generalization. In: Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2016). pp. 614–630. ACM (2016). https://doi.org/10.1145/2908080.2908118
22. Pnueli, A.: The temporal logic of programs. In: 18th Annual Symposium on Foundations of Computer Science (FOCS 1977). pp. 46–57. IEEE (1977). https://doi.org/10.1109/SFCS.1977.32
23. Solar-Lezama, A.: Program Synthesis by Sketching. Ph.d. dissertation, University of California, Berkeley (2008), `https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-177.html`
24. tla-connect Contributors: tla-connect: Rust ↔ TLA+ and Apalache integration. GitHub Repository / Crates.io (2024), `https://github.com/wiggum-cc/tla-connect`
25. Yang, J., Sun, Y., Wu, Y., Caridad, R., Yuan, Y., Yao, J., Lu, S., Pei, K.: Exverus: Verus proof repair via counterexample reasoning. In: Proceedings of the 43rd International Conference on Machine Learning (ICML 2026). PMLR / Microsoft Research (2026), `https://www.microsoft.com/en-us/research/publication/exverus-verus-proof-repair-via-counterexample-reasoning/`
26. Zeller, A., Hildebrandt, R.: Simplifying and isolating failure-inducing input. IEEE Transactions on Software Engineering **28**(2), 183–200 (2002). https://doi.org/10.1109/32.988498